EMBEDMATCH

Embedding Model Benchmarking Platform

Cloud Computing Project

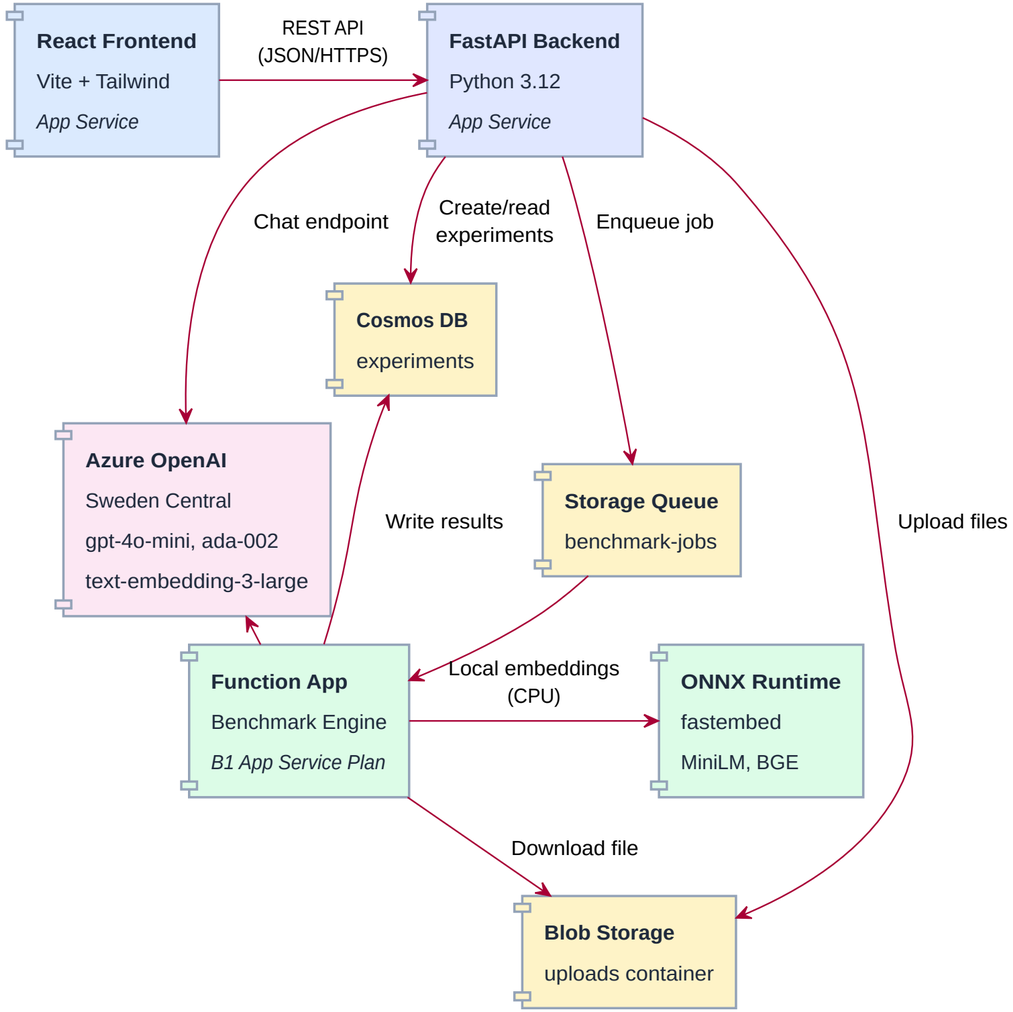
Jayasurya Marasani • Arunkumar Murugesan • Sneharajalakshmi Palanisamy • Jan Wojciech Wangrat

March 2026

WHAT IS EMBEDMATCH?

- A web platform to **compare embedding models** for text retrieval
- Users upload documents, select models, and run benchmarks
- Compares Azure OpenAI API models vs Open-Source models
- Measures: **retrieval quality**, **latency**, and **cost**

ARCHITECTURE OVERVIEW



AZURE RESOURCES

Service	Resource	Purpose
App Service	embed-match-web	FastAPI backend
App Service	embed-match-frontend	React frontend
Function App	embed-benchmark-fn-swe	Queue-triggered benchmark
Storage Account	embedstorage20147	Blob uploads + job queue
Cosmos DB	embed-db	Experiment results (NoSQL)
Azure OpenAI	embed-openai-sweden	GPT-4o-mini, ada-002, 3-large

Region: Switzerland North (Cosmos, Storage, App Services) • Sweden Central (OpenAI, Function App)

COMPONENT COMMUNICATION

1. UPLOAD FLOW

1. Frontend → `POST /uploads/`
2. Backend uploads file to **Blob Storage**
3. Returns blob name to frontend

2. START BENCHMARK

1. Frontend → `POST /experiments/`
2. Backend creates doc in **Cosmos DB**
3. Backend enqueues job to **Storage Queue**
4. Returns experiment ID

3. RUN BENCHMARK (ASYNC)

1. **Function App** dequeues message
2. Downloads blob from Storage
3. Generates queries via GPT-4o-mini
4. Embeds docs with each model
5. LLM-judges retrieval quality
6. Writes results to **Cosmos DB**

4. VIEW RESULTS

1. Frontend polls `GET /experiments/{id}`
2. Backend reads from Cosmos DB
3. Renders leaderboard when complete

EMBEDDING MODELS COMPARED

Model	Type	Dims	Cost/M tok	MTEB
Ada 002	API	1536	$0.10	61.0
Text Embedding 3 Large	API	3072	$0.13	64.6
MiniLM-L6-v2	Open Source	384	Free	56.3
BGE Small EN v1.5	Open Source	384	Free	62.2
BGE Base EN v1.5	Open Source	768	Free	63.6

Open-source models run locally on the Function App via **fastembed** (ONNX Runtime) — no external API calls, zero cost per token

DUAL-MODE EMBEDDING ENGINE

API MODELS

- Azure OpenAI SDK call
- ~6 ms per document
- Pay per token
- Models hosted by Microsoft
- Requires network round-trip

LOCAL MODELS

- fastembed / ONNX Runtime
- ~250 ms per document (CPU)
- Free (compute only)
- Models downloaded to Function App
- Single-model eviction to bound memory

ASYNCHRONOUS PROCESSING

WHY QUEUE-BASED?

- Benchmarks take 30–120s per model
- HTTP timeout would kill the request
- Queue decouples web tier from compute

FLOW

1. Backend enqueues experiment ID
2. Function App dequeues & processes
3. Results written to Cosmos DB
4. Frontend polls until complete

FAULT TOLERANCE

- Visibility timeout: 10 min (covers model download on first run)
- Max dequeue count: 5, then → poison queue
- Function auto-retries on transient failures
- **Progress tracking** — per-model updates written to Cosmos DB during benchmark

POISON QUEUE

- Failed messages go to `benchmark-jobs-poison`
- Can be re-enqueued or inspected manually
- Prevents infinite retry loops

COSMOS DB – NOSQL DOCUMENT STORE

Database: `embedbench` | **Container:** `experiments` | **Partition key:** `/id`

```
{
  "id": "bdc086b0-...",
  "status": "completed",          // pending -> running -> completed/failed
  "blob_name": "dataset.csv",
  "selected_models": ["text-embedding-ada-002", "BAAI/bge-small-en-v1.5"],
  "results": [
    { "model_name": "...", "retrieval_score": 0.80, "latency_ms": 6.2, ... },
    { "model_name": "...", "retrieval_score": 0.78, "latency_ms": 252, ... }
  ]
}
```

Why Cosmos? Flexible schema, global distribution ready, serverless pricing tier available

STUDENT LICENSE DRAWBACKS

REGION & RESOURCE RESTRICTIONS

- Many regions locked (US, Asia, Australia, Canada, Brazil, etc.)
- Some resource types unavailable in allowed regions (e.g., no Linux Consumption Plan in Switzerland North)
- Azure OpenAI quota per region — Switzerland North has **zero** chat model quota → had to deploy OpenAI in Sweden Central
- Limited TPM (tokens per minute) per model — throttles large benchmarks

NO RBAC (ROLE-BASED ACCESS CONTROL)

- Cannot create **Service Principals** → no passwordless CI/CD (OIDC)
- Cannot create **Managed Identities** → services authenticate via connection strings instead of identity
- No access to **Azure AD / Entra ID** app registrations → cannot add user authentication
- Cannot assign roles (e.g., "Storage Blob Data Contributor") → must use full connection strings with account keys
- Cannot set up **VNet integration** or **Private Endpoints** → must rely on IP whitelisting

SAMPLE BENCHMARK RESULTS

Model	Type	Score	Latency	Cost/M
BGE Small EN v1.5	Open Source	**0.70**	250 ms	Free
MiniLM-L6-v2	Open Source	0.62	240 ms	Free
Ada 002	API	0.50	6 ms	$0.10

LLM judge uses deterministic setup: `temperature=0`, `seed=42`, structured JSON counting prompt — 100% consistent across runs

TECHNOLOGY STACK

BACKEND

- Python 3.12, FastAPI
- Azure SDK (Storage, Cosmos, OpenAI)
- Pydantic for data validation
- uv for dependency management

FUNCTION APP

- Python 3.12, Azure Functions v2
- fastembed (ONNX Runtime)
- Queue trigger (event-driven)

FRONTEND

- React 18, TypeScript
- Vite build tool
- Tailwind CSS + shadcn/ui
- TanStack Query (data fetching)
- Recharts (visualizations)

DEVOPS

- GitHub Actions (3 pipelines)
- Kudu / Oryx / OneDeploy
- pnpm (frontend), uv (backend)

SUMMARY

- **9 Azure resources** working together via queues, REST APIs, and SDKs
- **True serverless FaaS** — Consumption Plan with scale-to-zero and auto-scaling
- **Event-driven architecture** — queue decouples long-running benchmarks from the web tier
- **Dual embedding engine** — API-based and local ONNX models in one platform
- **Deterministic LLM evaluation** — structured JSON counting for reproducible results
- **CI/CD** — automated for backend & frontend, manual for Function App
- **Student license workarounds** — creative solutions for real-world constraints

Thank you!